

Intermediate (measure the sequential version) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real measurement pipeline handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This exercise is the Basic exercise's "sequential" design, actually built — no new pattern, just two plain functions called one after the other, where the second one's input is the first one's output.

Before writing code, go re-read your own Basic-exercise prediction for the sequential design. You're about to find out if you were right.

Think about what "measuring" really means here: you need a number for cost (tokens or dollars), and a number for speed (seconds), for each stage separately and for the whole run.

Things to use:

- Two plain functions: `research(topic)` and `write(research_text)`, chained directly.
- `time.time()` around each call, and around the whole thing.
- Your model response's usage info — check `response.usage_metadata` (or your provider's equivalent) instead of guessing token counts.
- A printed report at the end showing both stages and the total.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Two functions, chained directly — `write(research(topic))` — is the entire structure. There's no graph here yet; that's deliberate, since a fixed two-step pipeline doesn't need LangGraph to prove the pattern works. (You'll build the graph version of a similar pipeline as part of the supervisor comparison and Project 4.)

What you're adding on top of the plain functions is measurement:

- **Wall-clock time** — wrap each call in `time.perf_counter()` before/after, and again around the whole thing. `time.perf_counter()` is meant specifically for measuring elapsed durations and isn't affected by system clock adjustments, unlike `time.time()`.
- **Token usage** — most LLM responses carry usage metadata (`response.usage_metadata` , a dict-like with `input_tokens` , `output_tokens` , `total_tokens`) — read it instead of estimating with a separate tokenizer; the provider already tells you exactly what it billed you for.
- **Keeping research's numbers separate from write's numbers** — you need both stages' individual counts, not just a combined total, or you can't tell which stage is actually the expensive one.

Sketch the two functions and where the timing/token-counting code goes around each one, before Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what happens the moment you want to compare this pipeline's numbers against something else — the supervisor version in `supervisor_compare`, or a different topic run tomorrow. A loose tuple of numbers, or three separately-returned values, gets confusing fast once you're comparing two runs side by side, and it's easy to accidentally compare research's tokens against write's seconds by mistake.

The real design question isn't just "measure each stage" — it's "what shape should these numbers live in, so a later comparison can't silently mix them up, and so the whole pipeline's report is reusable instead of one-off print statements?"

The extra pieces needed:

- A small typed result per stage (a `@dataclass` with `text`, `tokens`, `seconds` fields), instead of a raw tuple — so a caller reading `result.tokens` can't confuse it with `result.seconds`.
- A separate `run_sequential(topic)` function that owns the "run the whole pipeline and report on it" concern, distinct from what each stage itself does — so either the pipeline or an individual stage can be tested or reused alone.
- Thinking one step ahead to `supervisor_compare`: whatever shape you settle on here is the shape you'll want the supervisor version's report to match, field for field, so the two are directly comparable without translation.

Sketch this hardened version yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the two functions and the exact tools (`time`, `usage_metadata`) for a first working measurement. Intermediate explains why `perf_counter()` and real usage metadata matter, and why the two stages' numbers need to stay separate rather than only totaled. Advanced asks what shape those numbers should actually live in once they're not just going to be printed once and thrown away — a typed per-stage result and a dedicated report function — which is exactly the shape `supervisor_compare` needs to reuse for a fair, apples-to-apples comparison next.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
function research(topic):
    start timer
    call model to research the topic
    stop timer
    return text, tokens used, seconds taken

function write(research_text):
    start timer
    call model to write a summary from research_text
    stop timer
    return text, tokens used, seconds taken

main:
    research_result = research(topic)
    write_result = write(research_result.text)
    print each stage's tokens and seconds
    print the total tokens and total seconds
    compare against your Basic-exercise guess
```

Here's almost the whole thing — just try running it and reading it line by line:

```
import time

def research(topic):
    start = time.time()
    response = model.invoke("Research the top 3 causes of " + topic + ". List the
seconds = time.time() - start
tokens = response.usage_metadata["total_tokens"]
return response.content, tokens, seconds

def write(research_text):
    start = time.time()
    response = model.invoke("Write a 3-sentence summary of this:\n\n" + research_
seconds = time.time() - start
```

```
tokens = response.usage_metadata["total_tokens"]  
return response.content, tokens, seconds
```

Expected output if you run just this (nothing calls the functions yet): nothing — defining a function doesn't run it. Add `research_text, research_tokens, research_seconds = research("climate change")` below it, then call `write(research_text)` the same way, to see real numbers.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
def research(topic: str) -> dict:
    start = time.perf_counter()
    response = model.invoke(f"Research the top 3 causes of {topic}. List them bri
    elapsed = time.perf_counter() - start
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }

def write(research_text: str) -> dict:
    start = time.perf_counter()
    response = model.invoke(f"Write a 3-sentence summary of this:\n\n{research_te
    elapsed = time.perf_counter() - start
    return {
        "text": response.content,
        "tokens": response.usage_metadata["total_tokens"],
        "seconds": elapsed,
    }
```

```
def main():
    topic = "climate change"
    total_start = time.perf_counter()

    research_result = research(topic)
    write_result = write(research_result["text"])

    total_seconds = time.perf_counter() - total_start
    total_tokens = research_result["tokens"] + write_result["tokens"]

    print(f"research: {research_result['tokens']} tokens, {research_result['secon
    print(f"write: {write_result['tokens']} tokens, {write_result['seconds']:.2f}
    print(f"total: {total_tokens} tokens, {total_seconds:.2f}s")
```

Run it, then compare these real numbers against your Basic-exercise guess before checking Advanced.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
make a StageResult dataclass with fields: text, tokens, seconds

function research(topic) -> StageResult:
    time it, call the model, wrap the result in StageResult

function write(research_text) -> StageResult:
    time it, call the model, wrap the result in StageResult

function run_sequential(topic) -> dict:
    time the whole thing
    research_result = research(topic)
    write_result = write(research_result.text)
    return a report dict: research, write, total_tokens, total_seconds
```

Here's almost the whole thing — fill in the missing piece yourself:

```
import time
from dataclasses import dataclass

@dataclass
class StageResult:
    text: str
    tokens: int
    seconds: float

def research(topic: str) -> StageResult:
    start = time.perf_counter()
    response = model.invoke(f"Research the top 3 causes of {topic}. List them bri
    elapsed = time.perf_counter() - start
    return StageResult(response.content, response.usage_metadata["total_tokens"],

def write(research_text: str) -> StageResult:
    # your turn: same shape as research() above, but for the write stage
    ...
```

```
def run_sequential(topic: str) -> dict:
    total_start = time.perf_counter()
    research_result = research(topic)
    write_result = write(research_result.text)
    total_seconds = time.perf_counter() - total_start

    return {
        "research": research_result,
        "write": write_result,
        "total_tokens": research_result.tokens + write_result.tokens,
        "total_seconds": total_seconds,
    }
```

Fill in `write()` yourself, then compare your written reflection ("was I right about which stage costs more?") against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the same underlying shape (time it, call the model, read `usage_metadata`, report both stages and the total) at 3 completeness levels — Basic's version returns three loose values per function and prints once, Intermediate adds the type contract and a `main()` that separates "run" from "report," and Advanced adds a `StageResult` type plus a dedicated `run_sequential()` function, which is the reusable shape `supervisor_compare` needs next for a fair, side-by-side comparison.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)